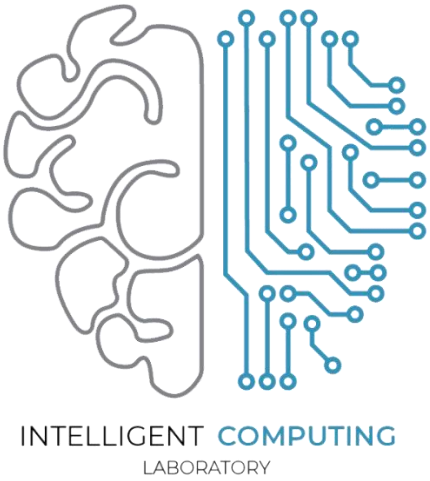
LAPORAN PRAKTIKUM

PEMBELAJARAN MESIN



NAMA : Purnomo Jeremy Bambang

NIM : 202431026

KELAS : C

DOSEN : Dr. Efy Yosrita, S.Si., M.Kom

NO.PC : 05

ASISTEN :

1. Muhammad Raka Ilham
2. Muhammad Hanif Nuruddin Jabbar
3. Muhammad Fachrydzi Irfansyah
4. Sharira Maharani

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2025/2026

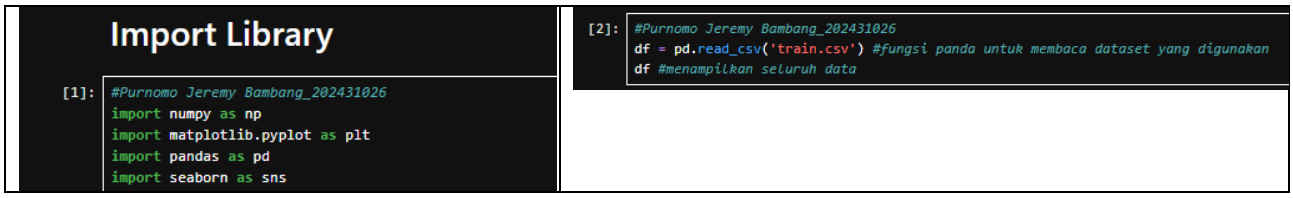
1. EDA

Judul dataset: Titanic - Machine Learning from Disaster

Link Kaggle: https://www.kaggle.com/competitions/titanic/data?select=train.csv

Jumlah baris × kolom: 891 baris × 12 kolom

Penjelasan data: Dataset ini berisi data penumpang kapal Titanic, seperti umur, jenis kelamin, kelas tiket, tarif, dan pelabuhan keberangkatan, beserta keterangan apakah penumpang selamat atau tidak. Datanya dipakai untuk mempelajari faktor apa saja yang berhubungan dengan keselamatan penumpang.

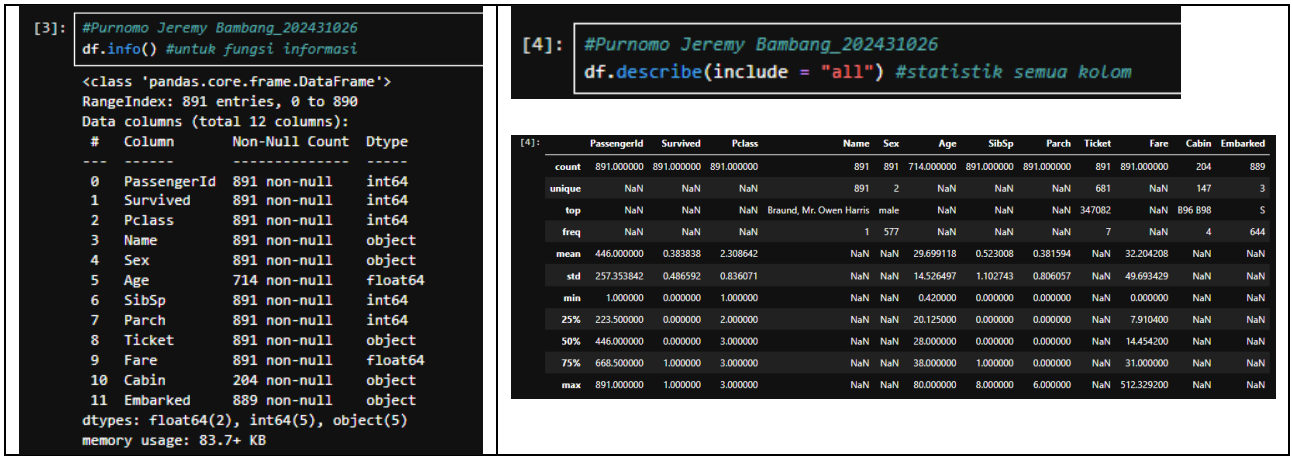


Cell 1: Import Library

Memanggil numpy, matplotlib, pandas, dan seaborn yang dipakai di seluruh program. Tidak ada output karena hanya menyiapkan library.

Cell 2: Baca dataset

pd.read_csv('train.csv') membaca file CSV ke variabel df, lalu df menampilkannya. Outputnya tabel data penumpang Titanic berukuran 891 baris × 12 kolom. Beberapa sel NaN (misalnya Cabin) menandakan data kosong.



[4]:

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked
count	891.000000	891.000000	891.000000	891	891	714.000000	891.000000	891.000000	891	891.000000	204	889
unique	NaN	NaN	NaN	891	2	NaN	NaN	NaN	681	NaN	147	3
top	NaN	NaN	NaN	Braund, Mr. Owen Harris	male	NaN	NaN	NaN	347082	NaN	B96 B98	S
freq	NaN	NaN	NaN	1	577	NaN	NaN	NaN	7	NaN	4	644
mean	446.000000	0.383838	2.308642	NaN	NaN	29.699118	0.523008	0.381594	NaN	32.204208	NaN	NaN
std	257.353842	0.486592	0.836071	NaN	NaN	14.526497	1.102743	0.806057	NaN	49.693429	NaN	NaN
min	1.000000	0.000000	1.000000	NaN	NaN	0.420000	0.000000	0.000000	NaN	0.000000	NaN	NaN
25%	223.500000	0.000000	2.000000	NaN	NaN	20.125000	0.000000	0.000000	NaN	7.910400	NaN	NaN
50%	446.000000	0.000000	3.000000	NaN	NaN	28.000000	0.000000	0.000000	NaN	14.454200	NaN	NaN
75%	668.500000	1.000000	3.000000	NaN	NaN	38.000000	1.000000	0.000000	NaN	31.000000	NaN	NaN
max	891.000000	1.000000	3.000000	NaN	NaN	80.000000	8.000000	6.000000	NaN	512.329200	NaN	NaN

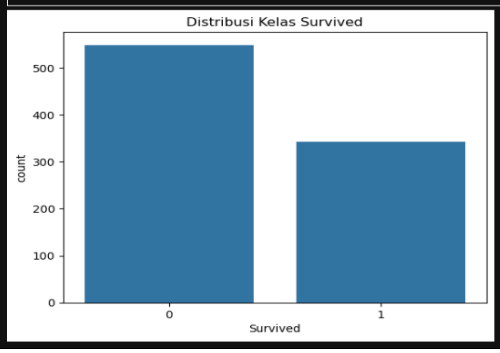
Cell 3: df.info()

Menampilkan jumlah baris, jumlah data terisi, dan tipe tiap kolom. Age hanya terisi 714, Cabin 204, dan Embarked 889 dari 891, sedangkan kolom lain terisi penuh.

Cell 4: df.describe(include="all")

Menampilkan statistik semua kolom. Rata-rata umur 29,70 tahun, rata-rata tarif 32,20, dan sekitar 38% penumpang selamat. Penumpang terbanyak laki-laki (577) dan pelabuhan terbanyak S (644).

```
[5]: #Purnomo Jeremy Bambang_202431026
sns.countplot(x = "Survived", data = df) #fungsi seaborn untuk menghitung penumpang selamat
plt.title("Distribusi Kelas Survived") #Judul
plt.show() #menampilkan hasil
```




```
[6]: df.dtypes #Purnomo Jeremy Bambang_202431026

[6]: PassengerId      int64
     Survived         int64
     Pclass           int64
     Name            object
     Sex             object
     Age            float64
     SibSp            int64
     Parch            int64
     Ticket          object
     Fare           float64
     Cabin           object
     Embarked        object
     dtype: object
```

Cell 5: Countplot Survived

Menghitung jumlah penumpang tiap nilai Survived. Batang 0 (tidak selamat) sekitar 550 orang, lebih tinggi daripada batang 1 (selamat) sekitar 340 orang.

Cell 6: df.dtypes

Menampilkan tipe tiap kolom. Name, Sex, Ticket, Cabin, dan Embarked masih bertipe object (teks).

```
[6]: #Purnomo Jeremy Bambang_202431026
cat_col = ["Sex", "Cabin", "Embarked"]

for col in cat_col:
    df[col] = df[col].astype('category')

df.dtypes

[6]: PassengerId      int64
     Survived         int64
     Pclass           int64
     Name            object
     Sex           category
     Age            float64
     SibSp            int64
     Parch            int64
     Ticket          object
     Fare           float64
     Cabin         category
     Embarked      category
     dtype: object
```

```
[7]: #Purnomo Jeremy Bambang_202431026
for col in cat_col:
    print(col, ":", df[col].unique())

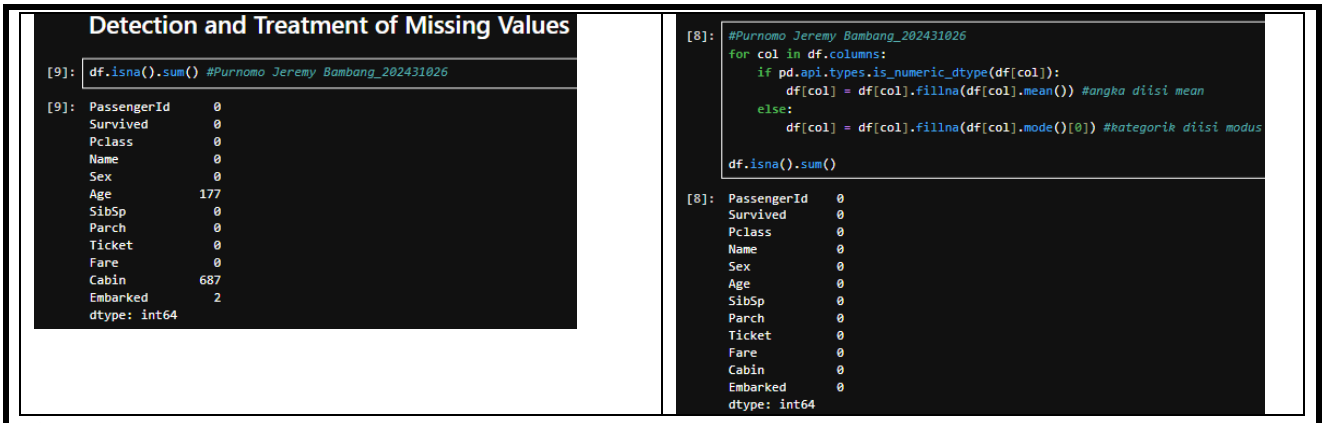
Sex : ['male', 'female']
Categories (2, object): ['female', 'male']
Cabin : [NaN, 'C85', 'C123', 'E46', 'G6', ..., 'E17', 'A24', 'C50', 'B42', 'C148']
Length: 148
Categories (147, object): ['A10', 'A14', 'A16', 'A19', ..., 'F38', 'F4', 'G6', 'T']
Embarked : ['S', 'C', 'Q', NaN]
Categories (3, object): ['C', 'Q', 'S']
```

Cell 7: Ubah ke category

Mengubah Sex, Cabin, dan Embarked menjadi tipe category lewat perulangan. Output dtypes sesudahnya menunjukkan ketiganya kini category.

Cell 8: Isi unik kolom kategorik

Mencetak nilai unik tiap kolom kategori. Sex berisi male dan female, Embarked berisi S, C, Q, dan Cabin punya 147 kategori. NaN di Cabin dan Embarked menunjukkan ada data kosong.

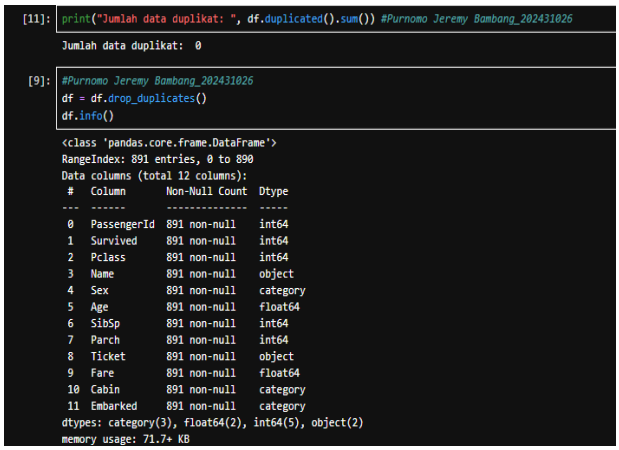


Cell 9: df.isna().sum()

Menghitung data kosong per kolom. Age 177, Cabin 687, Embarked 2, dan kolom lain 0.

Cell 10: Isi data kosong

Kolom angka diisi mean dan kolom non-angka diisi modus, hasilnya disimpan ke df. Output isna().sum() kini 0 semua. Age terisi rata-rata 29,70, Embarked terisi S, dan Cabin terisi B96 B98.



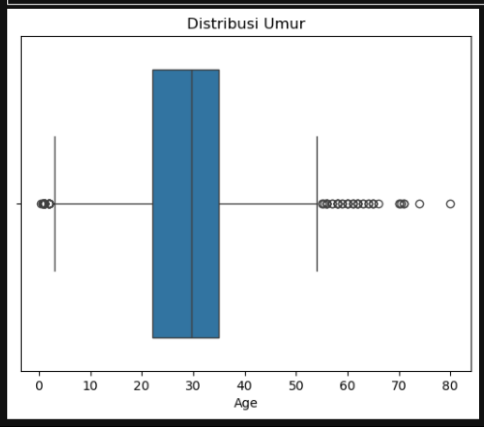
Cell 11: Hitung data duplikat

Menghitung baris yang sama persis dengan baris lain. Hasilnya 0, artinya tidak ada data duplikat.

Cell 12: Hapus duplikat dan info()

drop_duplicates() membuang duplikat, lalu info() memeriksa hasilnya. Tetap 891 baris karena tidak ada duplikat, dan semua kolom kini terisi penuh.

```
[10]: #Purnomo Jeremy Bambang_202431026
sns.boxplot(x="Age", data = df)
plt.title("Distribusi Umur")
plt.show()
```




```
[11]: #Purnomo Jeremy Bambang_202431026
Q1 = df['Age'].quantile(1/4) #quartile 1 = 25% data bawah
Q3 = df['Age'].quantile(3/4) #quartile 3 = 75% data

IQR = Q3-Q1

lower_bound = Q1 - 3/2 * IQR
upper_bound = Q3 + 3/2 * IQR

outliers = df[(df["Age"] < lower_bound) | (df["Age"] > upper_bound)]
print("Jumlah outlier umur: ", len(outliers))
```

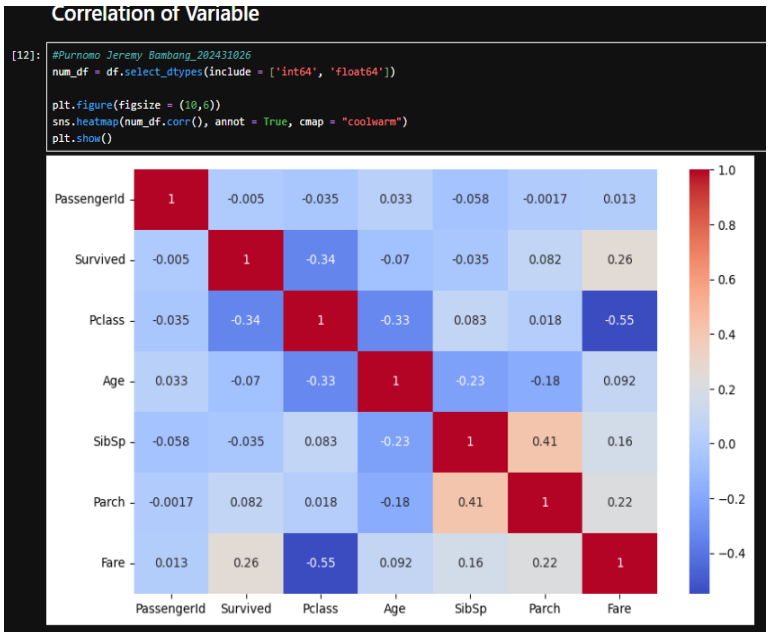
```
Jumlah outlier umur:  66
```

Cell 13: Boxplot Age

Menggambarkan sebaran umur. Kotak ada di sekitar umur 22–35 dengan median sekitar 30. Titik di kiri adalah bayi dan anak, titik di kanan adalah penumpang berumur di atas sekitar 54 tahun.

Cell 14: Hitung outlier dengan IQR

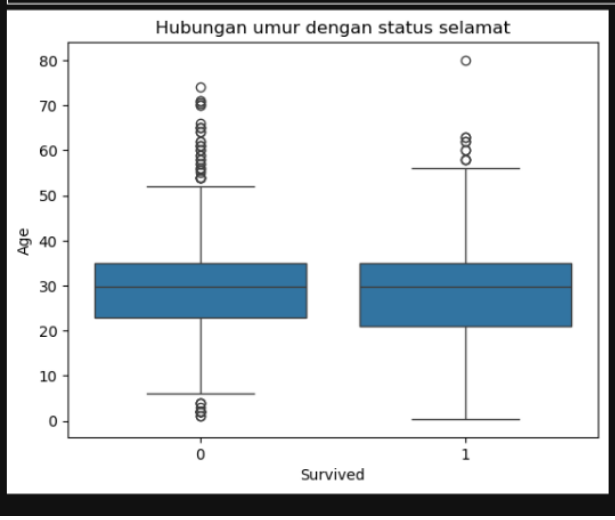
Umur di luar Q1 − 1,5×IQR dan Q3 + 1,5×IQR dihitung sebagai outlier. Hasilnya 66 outlier, sesuai titik-titik di luar kumis pada boxplot.



Cell 15: Heatmap korelasi

Menampilkan korelasi antar kolom angka (dekat 1 atau −1 berarti kuat, dekat 0 berarti lemah). Pclass dengan Fare −0,55 dan Survived dengan Pclass −0,34. Survived dengan Age hanya −0,07, jadi lemah.

```
[13]: #Purnomo Jeremy Bambang_202431026
sns.boxplot(x="Survived", y="Age", data = df)
plt.title("Hubungan umur dengan status selamat")
plt.show()
```

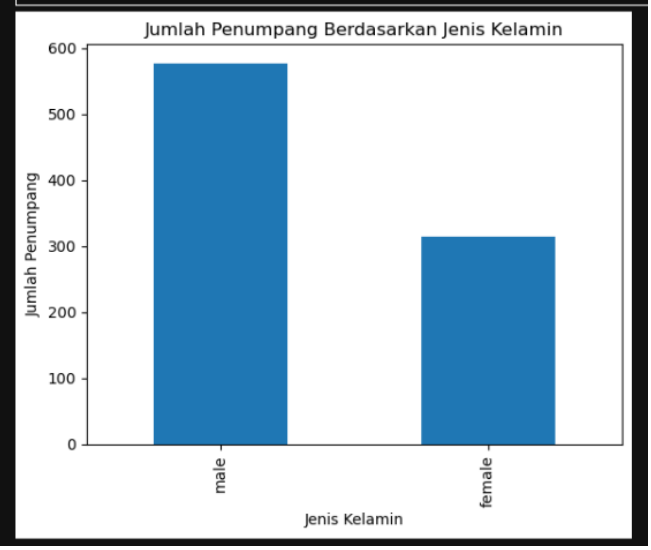


Cell 16: Boxplot Survived vs Age

Membandingkan sebaran umur penumpang yang tidak selamat (0) dan selamat (1). Kedua kotak mirip dengan median sekitar 30, jadi umur saja tidak membedakan keselamatan dengan jelas.

Visualisasi Jumlah Tiap Kategori

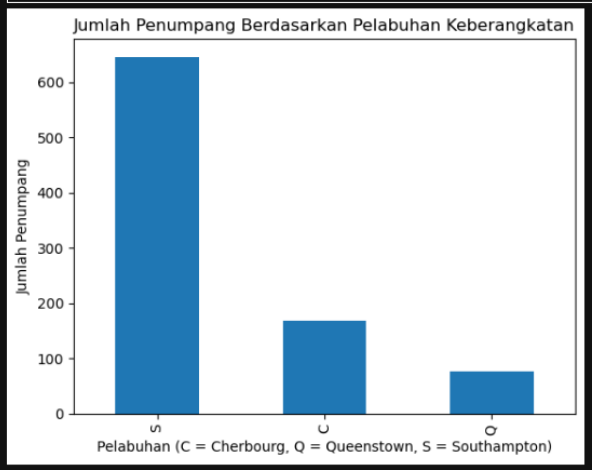
```
[14]: #Purnomo Jeremy Bambang_202431026
df['Sex'].value_counts().plot(kind='bar')
plt.title('Jumlah Penumpang Berdasarkan Jenis Kelamin')
plt.xlabel('Jenis Kelamin')
plt.ylabel('Jumlah Penumpang')
plt.show()
```



Cell 17: Bar chart jenis kelamin

value_counts() menghitung jumlah tiap kategori Sex, lalu digambar sebagai batang. Laki-laki sekitar 577 dan perempuan sekitar 314.

```
[15]: #Purnomo Jeremy Bambang_202431026
df['Embarked'].value_counts().plot(kind='bar')
plt.title('Jumlah Penumpang Berdasarkan Pelabuhan Keberangkatan')
plt.xlabel('Pelabuhan (C = Cherbourg, Q = Queenstown, S = Southampton)')
plt.ylabel('Jumlah Penumpang')
plt.show()
```



Cell 18: Bar chart pelabuhan keberangkatan

Cara kerjanya sama untuk Embarked. S terbanyak (sekitar 646), disusul C (sekitar 168) dan Q (sekitar 77).

2. Scraping

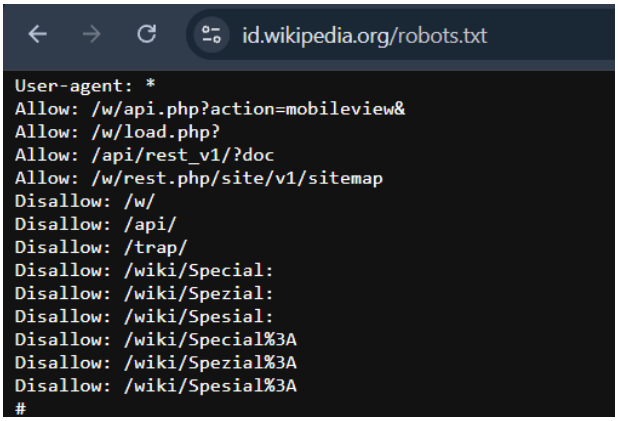
Situs yang digunakan: Wikipedia Indonesia (https://id.wikipedia.org). Halaman yang di-scrape adalah halaman HTML biasa, bukan API.

Link halaman yang di-scrape:

- Daftar kabupaten: https://id.wikipedia.org/wiki/Daftar_kabupaten_di_Indonesia
- Daftar kota: https://id.wikipedia.org/wiki/Daftar_kota_di_Indonesia

robots.txt

- Alamat robots.txt: https://id.wikipedia.org/robots.txt
- Screenshot isi robots.txt:



- Aturan yang berkaitan dengan halaman yang diambil: Pada blok User-agent: * (berlaku untuk semua klien), path yang dilarang adalah /w/, /api/, /trap/, serta halaman khusus /wiki/Special:, /wiki/Spezial:, dan /wiki/Spesial: (termasuk varian %3A-nya).

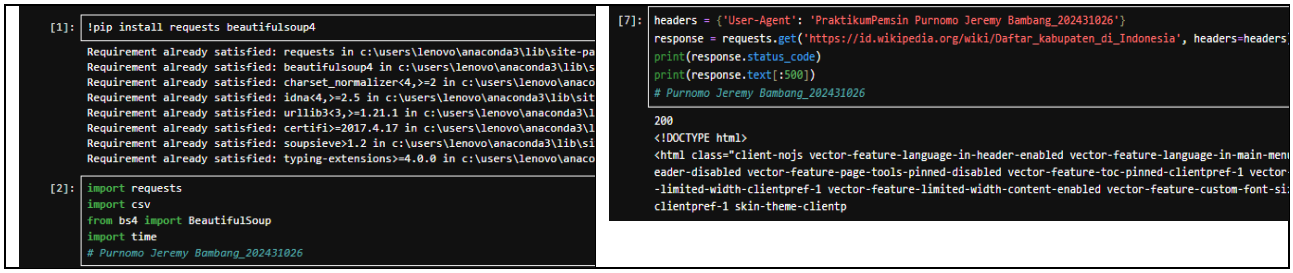
- Alasan halaman boleh diambil: Halaman yang saya ambil adalah /wiki/Daftar_kabupaten_di_Indonesia dan /wiki/Daftar_kota_di_Indonesia. Path tersebut tidak diawali satu pun path yang dilarang di atas, sehingga tidak ada aturan Disallow yang mengenainya. Saya mengambil halaman HTML biasa, tidak memakai /api/ maupun /w/. Saya juga memberi jeda 2 detik antar-permintaan dan mencantumkan User-Agent yang menyebut nama program dan identitas saya.

Data yang diperoleh

- Jumlah data: 514 data (416 kabupaten dan 98 kota)
- Atribut angka: kecamatan, kelurahan, desa (jumlah kecamatan, kelurahan, dan desa pada tiap daerah)

Atribut kategori: provinsi dan jenis (Kabupaten atau Kota)

Kegunaan data: Data ini dipakai pada praktikum Regresi Linear (memprediksi jumlah desa dari jumlah kecamatan dan kelurahan) dan Decision Tree (memprediksi jenis daerah, kabupaten atau kota, dari jumlah kecamatan, kelurahan, dan desa).



Cell 1: Install library

!pip install requests beautifulsoup4 memastikan library yang dibutuhkan sudah terpasang. Output Requirement already satisfied menandakan keduanya memang sudah ada di komputer.

Cell 2: Import library

Memanggil requests untuk mengambil halaman web, csv untuk menyimpan hasil, BeautifulSoup untuk membaca HTML, dan time untuk memberi jeda antar-permintaan. Tidak ada output.

Cell 3: Ambil halaman web

headers berisi User-Agent untuk memperkenalkan program ke Wikipedia. requests.get mengambil halaman daftar kabupaten, lalu status_code dicetak. Outputnya 200, yang berarti halaman berhasil diambil, diikuti 500 karakter awal kode HTML halaman tersebut.

```
[8]: soup = BeautifulSoup(response.text, "html.parser")
     tabel = soup.find("table", class_="wikitable")
     blocks = tabel.find_all("tr")[2:] #2 baris pertama adalah header
     print(f"kabupaten yang ditemukan pada website ini sejumlah: {len(blocks)}")
     # Purnomo Jeremy Bambang_202431026

     kabupaten yang ditemukan pada website ini sejumlah: 416
```

```
[9]: result = [] #Purnomo Jeremy Bambang_202431026
     for block in blocks:
         kolom = block.find_all(["th", "td"])
         if len(kolom) != 8: #baris kabupaten harus punya 8 sel
             continue
         nama = kolom[1].get_text(strip=True)
         provinsi = kolom[3].get_text(strip=True)
         kecamatan = kolom[5].get_text(strip=True)
         kelurahan = kolom[6].get_text(strip=True)
         desa = kolom[7].get_text(strip=True)

         result.append({"nama": nama, "provinsi": provinsi, "kecamatan": kecamatan,
                        "kelurahan": kelurahan, "desa": desa, "jenis": "Kabupaten"})

     print(len(result), "data masuk,", len(blocks) - len(result), "baris dilewati")

     416 data masuk, 0 baris dilewati
```

Cell 4: Cari blok data

Teks HTML diubah jadi objek BeautifulSoup agar bisa dicari. find mengambil tabel data (wikitable), dan find_all("tr")[2:] mengambil semua baris datanya dengan melewati 2 baris header. Outputnya: 416 kabupaten ditemukan, melebihi syarat minimal 200.

Cell 5: Ambil atribut kabupaten

Tiap baris dibaca selnya (th dan td). Baris yang tidak punya 8 sel dilewati. Nama, provinsi, kecamatan, kelurahan, dan desa diambil dengan get_text(strip=True), lalu disimpan ke list result sebagai kamus dengan jenis “Kabupaten”. Outputnya: 416 data masuk, 0 baris dilewati, artinya semua baris berhasil diambil.

```
[10]: time.sleep(2)
      response = requests.get('https://id.wikipedia.org/wiki/Daftar_kota_di_Indonesia', headers=headers)
      print(response.status_code)

      soup = BeautifulSoup(response.text, "html.parser")
      tabel = soup.find("table", class_="wikitable")
      blocks = tabel.find_all("tr")[2:]
      print(f"kota yang ditemukan pada website ini sejumlah: {len(blocks)}")

      jumlah_kab = len(result)
      for block in blocks:
          kolom = block.find_all(["th", "td"])
          if len(kolom) != 7: #baris kota tidak punya kolom ibu kota, jadi 7 sel
              continue
          nama = kolom[1].get_text(strip=True)
          provinsi = kolom[3].get_text(strip=True)
          kecamatan = kolom[4].get_text(strip=True)
          kelurahan = kolom[5].get_text(strip=True)
          desa = kolom[6].get_text(strip=True)

          result.append({"nama": nama, "provinsi": provinsi, "kecamatan": kecamatan,
                         "kelurahan": kelurahan, "desa": desa, "jenis": "Kota"})

      print(len(result) - jumlah_kab, "data kota masuk,", len(blocks) - (len(result) - jumlah_kab), "baris dilewati")
      print("Total data:", len(result))
      # Purnomo Jeremy Bambang_202431026

      200
      kota yang ditemukan pada website ini sejumlah: 98
      98 data kota masuk, 0 baris dilewati
      Total data: 514
```

Cell 6: Ulangi untuk halaman kota

Langkah yang sama diulang untuk halaman daftar kota, setelah jeda 2 detik. Bedanya, baris kota hanya punya 7 sel (tidak ada kolom ibu kota), jadi nomor kolomnya bergeser, dan jenis diisi “Kota”. Outputnya: status 200, 98 kota ditemukan, 98 data masuk, 0 dilewati, dan Total data: 514.

```
[7]: for item in result:
         print(f"{item['jenis']}: {item['nama']} - provinsi: {item['provinsi']} - kecamatan: {item['kecamatan']} - kelurahan: {item['kelurahan']} - desa: {i

     # Purnomo Jeremy Bambang_202431026
```

```
Kabupaten: Aceh Barat - provinsi: Aceh - kecamatan: 12 - kelurahan: 0 - desa: 322
Kabupaten: Aceh Barat Daya - provinsi: Aceh - kecamatan: 9 - kelurahan: 0 - desa: 152
Kabupaten: Aceh Besar - provinsi: Aceh - kecamatan: 23 - kelurahan: 0 - desa: 604
Kabupaten: Aceh Jaya - provinsi: Aceh - kecamatan: 9 - kelurahan: 0 - desa: 172
Kabupaten: Aceh Selatan - provinsi: Aceh - kecamatan: 18 - kelurahan: 0 - desa: 260
Kabupaten: Aceh Singkil - provinsi: Aceh - kecamatan: 11 - kelurahan: 0 - desa: 116
Kabupaten: Aceh Tamiang - provinsi: Aceh - kecamatan: 12 - kelurahan: 0 - desa: 216
Kabupaten: Aceh Tengah - provinsi: Aceh - kecamatan: 14 - kelurahan: 0 - desa: 295
Kabupaten: Aceh Tenggara - provinsi: Aceh - kecamatan: 16 - kelurahan: 0 - desa: 385
Kabupaten: Aceh Timur - provinsi: Aceh - kecamatan: 24 - kelurahan: 0 - desa: 513
Kabupaten: Aceh Utara - provinsi: Aceh - kecamatan: 27 - kelurahan: 0 - desa: 852
Kabupaten: Agam - provinsi: Sumatera Barat - kecamatan: 16 - kelurahan: 0 - desa: 92
Kabupaten: Alor - provinsi: Nusa Tenggara Timur - kecamatan: 18 - kelurahan: 17 - desa: 158
Kabupaten: Asahan - provinsi: Sumatera Utara - kecamatan: 25 - kelurahan: 27 - desa: 177
Kabupaten: Asmat - provinsi: Papua Selatan - kecamatan: 25 - kelurahan: 0 - desa: 224
Kabupaten: Badung - provinsi: Bali - kecamatan: 6 - kelurahan: 16 - desa: 46
Kabupaten: Balangan - provinsi: Kalimantan Selatan - kecamatan: 8 - kelurahan: 3 - desa: 154
Kabupaten: Bandung - provinsi: Jawa Barat - kecamatan: 31 - kelurahan: 10 - desa: 270
```

Cell 7: Tampilkan dengan perulangan for

Setiap data di result dicetak satu per satu dalam satu kalimat berisi jenis, nama, provinsi, kecamatan, kelurahan, dan desa. Outputnya daftar 514 baris data hasil scraping.

```
[8]: with open("wilayah_indonesia.csv", "w", newline="", encoding="utf-8") as csvfile:
         fieldnames = ["nama", "provinsi", "kecamatan", "kelurahan", "desa", "jenis"]
         writer = csv.DictWriter(csvfile, fieldnames=fieldnames)

         writer.writeheader()
         for item in result:
             writer.writerow(item)

     # Purnomo Jeremy Bambang_202431026
```

Cell 8: Simpan ke CSV

csv.DictWriter menulis nama kolom lewat writeheader(), lalu setiap data lewat writerow(), ke file wilayah_indonesia.csv. Tidak ada output, tetapi file CSV terbentuk di folder notebook.

```
[9]: import pandas as pd # Purnomo Jeremy Bambang_202431026

[10]: db = pd.read_csv('wilayah_indonesia.csv', dtype=str) # Purnomo Jeremy Bambang_202431026
```

Cell 9: Import pandas

Memanggil pandas untuk membaca file CSV yang baru dibuat. Tidak ada output.

Cell 10: Baca CSV

pd.read_csv(..., dtype=str) membaca file CSV dengan semua kolom dibaca sebagai teks. Tidak ada output.

```
[11]: db # Purnomo Jeremy Bambang_202431026
```

[11]:

	nama	provinsi	kecamatan	kelurahan	desa	jenis
0	Aceh Barat	Aceh	12	0	322	Kabupaten
1	Aceh Barat Daya	Aceh	9	0	152	Kabupaten
2	Aceh Besar	Aceh	23	0	604	Kabupaten
3	Aceh Jaya	Aceh	9	0	172	Kabupaten
4	Aceh Selatan	Aceh	18	0	260	Kabupaten
...	...	...	...	...	...	...
509	Ternate	Maluku Utara	8	78	0	Kota
510	Tidore Kepulauan	Maluku Utara	8	40	49	Kota
511	Tomohon	Sulawesi Utara	5	44	0	Kota
512	Tual	Maluku	5	3	27	Kota
513	Yogyakarta	Daerah Istimewa Yogyakarta	14	45	0	Kota

514 rows × 6 columns

Cell 11: Tampilkan data

Menampilkan tabel hasil pembacaan. Outputnya tabel 514 baris × 6 kolom. Contohnya Aceh Barat (Kabupaten) punya 12 kecamatan, 0 kelurahan, dan 322 desa, sedangkan Ternate (Kota) punya 8 kecamatan, 78 kelurahan, dan 0 desa. Ini menunjukkan pola yang berbeda antara kabupaten dan kota.

```
[12]: db.info() # Purnomo Jeremy Bambang_202431026
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 514 entries, 0 to 513
Data columns (total 6 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   nama       514 non-null    object
 1   provinsi   514 non-null    object
 2   kecamatan  514 non-null    object
 3   kelurahan  514 non-null    object
 4   desa       514 non-null    object
 5   jenis      514 non-null    object
dtypes: object(6)
memory usage: 24.2+ KB
```

Cell 12: db.info() sebelum diubah

Menampilkan tipe tiap kolom. Outputnya semua kolom bertipe object (teks), termasuk kecamatan, kelurahan, dan desa. Ini menunjukkan data hasil scraping awalnya berupa teks sehingga perlu diubah.

Cell 13: Ubah kolom angka

Untuk kolom kecamatan, kelurahan, dan desa, simbol selain angka dibuang lalu pd.to_numeric mengubahnya menjadi angka. Hasilnya disimpan lagi ke file CSV. Tidak ada output.

```
[13]: for kolom in ["kecamatan", "kelurahan", "desa"]:
          db[kolom] = db[kolom].str.replace(r"[^\d]", "", regex=True) #buang catatan kaki/simbol
          db[kolom] = pd.to_numeric(db[kolom], errors="coerce")

      db.to_csv("wilayah_indonesia.csv", index=False)
      # Purnomo Jeremy Bambang_202431026
```

```
[14]: db = pd.read_csv('wilayah_indonesia.csv')
      db.info() # Purnomo Jeremy Bambang_202431026
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 514 entries, 0 to 513
Data columns (total 6 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   nama       514 non-null    object
 1   provinsi   514 non-null    object
 2   kecamatan  514 non-null    int64
 3   kelurahan  514 non-null    int64
 4   desa       514 non-null    int64
 5   jenis      514 non-null    object
dtypes: int64(3), object(3)
memory usage: 24.2+ KB
```

Cell 14: db.info() setelah diubah

Membaca ulang CSV dan memeriksa tipenya. Outputnya kecamatan, kelurahan, dan desa sekarang bertipe int64, sedangkan nama, provinsi, dan jenis tetap object. Semua kolom terisi penuh (514 non-null). Atribut angka sudah siap dipakai untuk Regresi Linear dan Decision Tree.

3. Regresi Linear

```
[1]: #Purnomo Jeremy Bambang_202431026
     import numpy as np
     import pandas as pd
     import matplotlib.pyplot as plt
     import seaborn as sns
     from sklearn.model_selection import train_test_split
     from sklearn.linear_model import LinearRegression
     from sklearn.metrics import mean_squared_error, r2_score
```

```
[2]: #Purnomo Jeremy Bambang_202431026
     df = pd.read_csv("wilayah_indonesia.csv")
     df
```

[2]:

	nama	provinsi	kecamatan	kelurahan	desa	jenis
0	Aceh Barat	Aceh	12	0	322	Kabupaten
1	Aceh Barat Daya	Aceh	9	0	152	Kabupaten
2	Aceh Besar	Aceh	23	0	604	Kabupaten
3	Aceh Jaya	Aceh	9	0	172	Kabupaten
4	Aceh Selatan	Aceh	18	0	260	Kabupaten
...	...	...	...	...	...	...
509	Ternate	Maluku Utara	8	78	0	Kota
510	Tidore Kepulauan	Maluku Utara	8	40	49	Kota
511	Tomohon	Sulawesi Utara	5	44	0	Kota
512	Tual	Maluku	5	3	27	Kota
513	Yogyakarta	Daerah Istimewa Yogyakarta	14	45	0	Kota

514 rows × 6 columns

Cell 1: Import library

Memanggil numpy dan pandas untuk mengolah data, matplotlib dan seaborn untuk grafik, train_test_split untuk membagi data, LinearRegression sebagai model, serta mean_squared_error dan r2_score untuk menilai model. Tidak ada output.

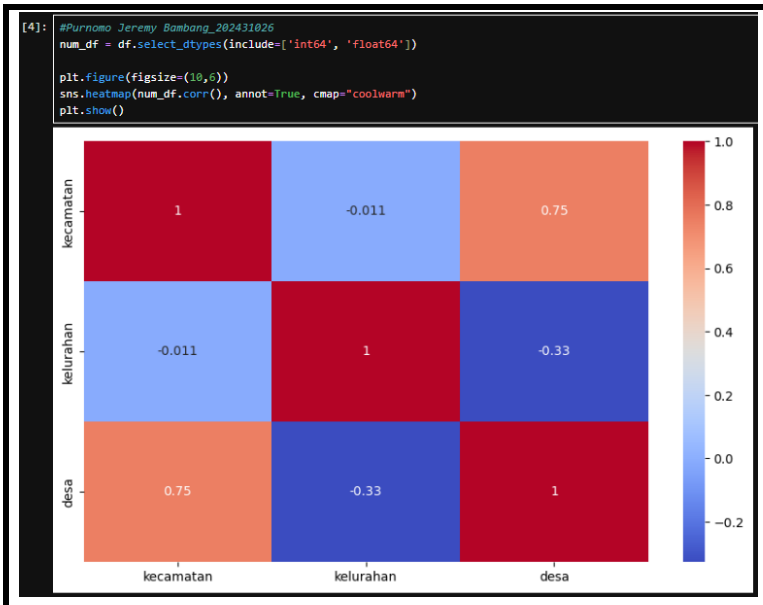
Cell 2: Baca dataset

pd.read_csv("wilayah_indonesia.csv") membaca data hasil scraping ke df, lalu df menampilkannya. Outputnya tabel 514 baris × 6 kolom. Contohnya Aceh Barat punya 12 kecamatan, 0 kelurahan, dan 322 desa.

```
[3]: df.info() #Purnomo Jeremy Bambang_202431026
     <class 'pandas.core.frame.DataFrame'>
     RangeIndex: 514 entries, 0 to 513
     Data columns (total 6 columns):
      #   Column     Non-Null Count  Dtype
     ---  ------     --------------  -----
      0   nama       514 non-null    object
      1   provinsi   514 non-null    object
      2   kecamatan  514 non-null    int64
      3   kelurahan  514 non-null    int64
      4   desa       514 non-null    int64
      5   jenis      514 non-null    object
     dtypes: int64(3), object(3)
     memory usage: 24.2+ KB
```

Cell 3: df.info()

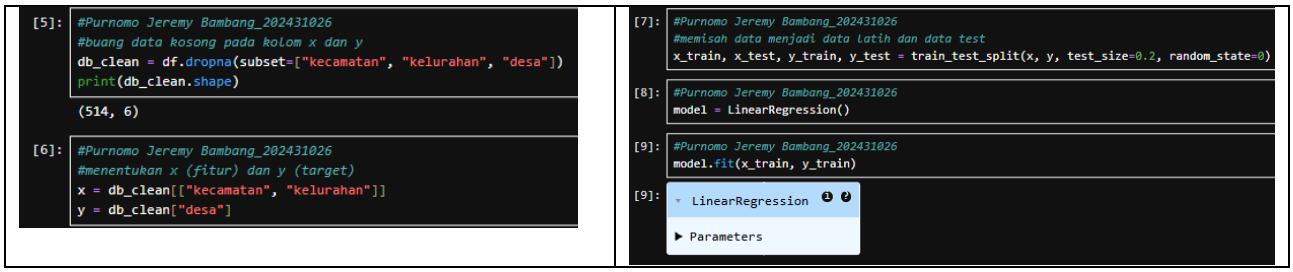
Menampilkan struktur data. Ada 514 baris, tidak ada data kosong, dan kecamatan, kelurahan, desa bertipe int64. Ini menunjukkan ketiganya atribut angka yang bisa dipakai untuk regresi.



Cell 4: Heatmap korelasi

- Mengambil kolom angka lalu menampilkan korelasinya dalam peta warna. Yang terlihat:
- kecamatan dengan desa bernilai 0,75, korelasi positif kuat. Makin banyak kecamatan, makin banyak desa.
- kelurahan dengan desa bernilai −0,33, korelasi negatif sedang. Daerah dengan banyak kelurahan cenderung punya lebih sedikit desa (ciri daerah kota).
- kecamatan dengan kelurahan hanya −0,011, hampir tidak berhubungan.

Hasil ini mendukung pemilihan kecamatan dan kelurahan sebagai fitur untuk memprediksi desa.



Cell 5: Buang data kosong

dropna(subset=[...]) membuang baris yang kolom x atau y-nya kosong. Outputnya (514, 6), ukurannya tidak berubah karena tidak ada data kosong.

Cell 6: Tentukan x dan y

x berisi fitur (kecamatan dan kelurahan), dan y berisi target (desa). Tidak ada output.

Cell 7: Bagi data latih dan uji

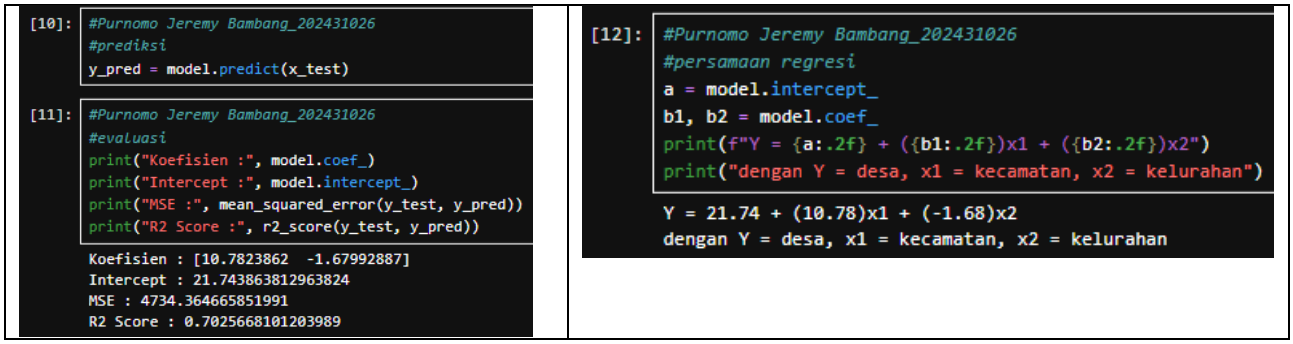
Data dibagi 80% untuk melatih dan 20% untuk menguji (test_size=0.2). random_state=0 membuat pembagiannya selalu sama tiap dijalankan. Tidak ada output.

Cell 8: Buat model

LinearRegression() membuat model regresi linear yang masih kosong dan belum belajar apa-apa. Tidak ada output.

Cell 9: Latih model

model.fit(x_train, y_train) membuat model mencari garis terbaik dari data latih. Outputnya LinearRegression(), tanda model sudah dilatih.



```
[10]: #Purnomo Jeremy Bambang_202431026
      #prediksi
      y_pred = model.predict(x_test)

[11]: #Purnomo Jeremy Bambang_202431026
      #evaluasi
      print("Koefisien :", model.coef_)
      print("Intercept :", model.intercept_)
      print("MSE :", mean_squared_error(y_test, y_pred))
      print("R2 Score :", r2_score(y_test, y_pred))

      Koefisien : [10.7823862  -1.67992887]
      Intercept : 21.743863812963824
      MSE : 4734.364665851991
      R2 Score : 0.7025668101203989
```

```
[12]: #Purnomo Jeremy Bambang_202431026
      #persamaan regresi
      a = model.intercept_
      b1, b2 = model.coef_
      print(f"Y = {a:.2f} + ({b1:.2f})x1 + ({b2:.2f})x2")
      print("dengan Y = desa, x1 = kecamatan, x2 = kelurahan")

      Y = 21.74 + (10.78)x1 + (-1.68)x2
      dengan Y = desa, x1 = kecamatan, x2 = kelurahan
```

Cell 10: Prediksi data uji

model.predict(x_test) menebak jumlah desa untuk data uji dan hasilnya disimpan di y_pred. Tidak ada output, tetapi hasil tebakan ini nanti dibandingkan dengan nilai asli.

Cell 11: Evaluasi model

Outputnya:

- Koefisien [10,78, −1,68]: tiap tambahan 1 kecamatan menaikkan prediksi desa sekitar 10,78, sedangkan tiap tambahan 1 kelurahan menurunkannya sekitar 1,68.
- Intercept 21,74: nilai dasar prediksi saat kecamatan dan kelurahan bernilai 0.
- MSE 4734,36: rata-rata kuadrat selisih prediksi dan nilai asli. Makin kecil makin baik.
- R^2 0,70: model mampu menjelaskan sekitar 70% variasi jumlah desa, termasuk baik.

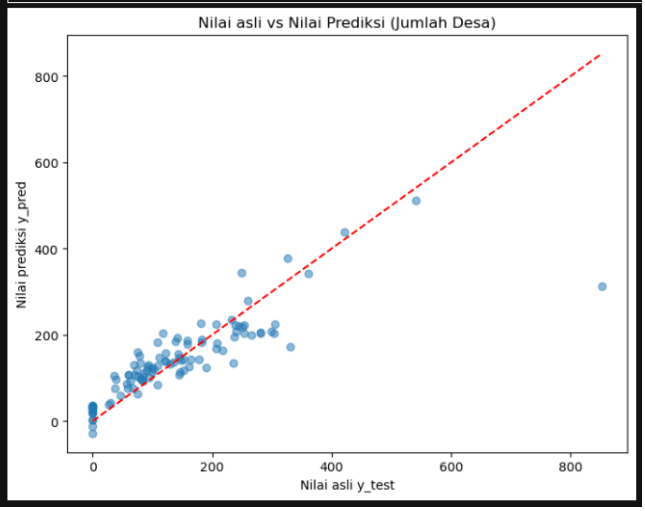
Cell 12: Persamaan regresi

Nilai intercept dan koefisien disusun menjadi persamaan:

$Y = 21{,}74 + (10{,}78)x1 + (-1{,}68)x2$

dengan Y = jumlah desa, x1 = jumlah kecamatan, x2 = jumlah kelurahan. Persamaan ini bisa dipakai menghitung perkiraan jumlah desa dari jumlah kecamatan dan kelurahan.

```
[13]: #Purnomo Jeremy Bambang_202431026
      ##visualisasi model
      plt.figure(figsize=(8,6))
      plt.scatter(y_test, y_pred, alpha=0.5)
      plt.xlabel("Nilai asli y_test")
      plt.ylabel("Nilai prediksi y_pred")
      plt.title("Nilai asli vs Nilai Prediksi (Jumlah Desa)")
      plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
      plt.show()
```



Cell 13: Visualisasi nilai asli vs prediksi

Scatter plot membandingkan nilai asli (sumbu x) dengan nilai prediksi (sumbu y), dengan garis merah putus-putus sebagai batas prediksi sempurna. Sebagian besar titik berada dekat garis merah, sehingga prediksi cukup mendekati nilai asli. Ada dua catatan:

Ada beberapa titik di kiri bawah yang prediksinya sedikit di bawah 0 atau di atas 0 padahal nilai aslinya 0. Itu daerah kota tanpa desa.

Ada satu titik jauh di kanan (nilai asli sekitar 850 desa) yang diprediksi hanya sekitar 310, yaitu daerah dengan jumlah desa luar biasa banyak yang sulit ditebak model.

4. Decision Tree

```
[1]: #Purnomo Jeremy Bambang_202431026
     import pandas as pd
     import numpy as np
     import matplotlib.pyplot as plt
```

PRE PROCESSING

```
[2]: #Purnomo Jeremy Bambang_202431026
     df = pd.read_csv('wilayah_indonesia.csv', sep=",")
     df
```

[2]:

	nama	provinsi	kecamatan	kelurahan	desa	jenis
0	Aceh Barat	Aceh	12	0	322	Kabupaten
1	Aceh Barat Daya	Aceh	9	0	152	Kabupaten
2	Aceh Besar	Aceh	23	0	604	Kabupaten
3	Aceh Jaya	Aceh	9	0	172	Kabupaten
4	Aceh Selatan	Aceh	18	0	260	Kabupaten
...	...	...	...	...	...	...
509	Ternate	Maluku Utara	8	78	0	Kota
510	Tidore Kepulauan	Maluku Utara	8	40	49	Kota
511	Tomohon	Sulawesi Utara	5	44	0	Kota
512	Tual	Maluku	5	3	27	Kota
513	Yogyakarta	Daerah Istimewa Yogyakarta	14	45	0	Kota

514 rows × 6 columns

Cell 1: Import library

Memanggil pandas untuk mengolah data, numpy untuk array angka, dan matplotlib untuk menampilkan pohon. Tidak ada output.

Cell 2: Baca dataset

pd.read_csv('wilayah_indonesia.csv') membaca data hasil scraping ke df, lalu df menampilkannya. Outputnya tabel 514 baris × 6 kolom, misalnya Aceh Barat (Kabupaten) punya 12 kecamatan, 0 kelurahan, dan 322 desa.

```
[3]: #Purnomo Jeremy Bambang_202431026
     df.info()

     <class 'pandas.core.frame.DataFrame'>
     RangeIndex: 514 entries, 0 to 513
     Data columns (total 6 columns):
      #   Column     Non-Null Count  Dtype
     ---  ------     --------------  -----
      0   nama       514 non-null    object
      1   provinsi   514 non-null    object
      2   kecamatan  514 non-null    int64
      3   kelurahan  514 non-null    int64
      4   desa       514 non-null    int64
      5   jenis      514 non-null    object
     dtypes: int64(3), object(3)
     memory usage: 24.2+ KB
```

```
[4]: #Purnomo Jeremy Bambang_202431026
     df.describe()
```

[4]:

	kecamatan	kelurahan	desa
count	514.000000	514.000000	514.000000
mean	14.151751	16.570039	146.429961
std	8.118297	23.215659	122.005065
min	2.000000	0.000000	0.000000
25%	8.000000	3.000000	61.250000
50%	12.000000	9.000000	125.000000
75%	19.000000	21.000000	215.750000
max	51.000000	177.000000	852.000000

Cell 3: df.info()

Menampilkan struktur data. Ada 514 baris tanpa data kosong. kecamatan, kelurahan, dan desa bertipe int64 (angka), sedangkan nama, provinsi, dan jenis bertipe object (teks).

Cell 4: df.describe()

Menampilkan statistik kolom angka. Rata-rata tiap daerah punya sekitar 14 kecamatan, 17 kelurahan, dan 146 desa. Kelurahan paling banyak 177 dan desa paling banyak 852, jadi rentang datanya lebar.

```
[5]: #Purnomo Jeremy Bambang_202431026
     df.isna().sum()

[5]: nama        0
     provinsi    0
     kecamatan   0
     kelurahan   0
     desa        0
     jenis       0
     dtype: int64

[15]: #Purnomo Jeremy Bambang_202431026
      df['jenis'].unique()

[15]: array(['Kabupaten', 'Kota'], dtype=object)
```

```
Modelling

[7]: #Purnomo Jeremy Bambang_202431026
     x = df.drop(['nama', 'provinsi', 'jenis'], axis=1)
     y = df['jenis']

[8]: #Purnomo Jeremy Bambang_202431026
     from sklearn.preprocessing import LabelEncoder

     le = LabelEncoder()
     y = le.fit_transform(y)

     for i, label in enumerate(le.classes_):
         print(f"{i} = {label}")

     x = np.array(x, dtype='float64')
     y = np.array(y, dtype='int64')

     0 = Kabupaten
     1 = Kota
```

Cell 5: df.isna().sum()

Menghitung data kosong per kolom. Semua kolom bernilai 0, artinya tidak ada data kosong dan data siap dipakai tanpa dibersihkan.

Cell 6: Isi unik kolom target

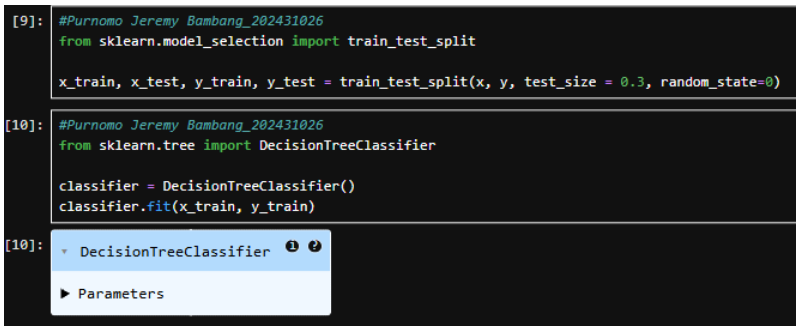
df['jenis'].unique() menampilkan kelas pada target. Outputnya ['Kabupaten', 'Kota']. Arti tiap kelas ditulis sebagai komentar di cell: Kabupaten umumnya banyak desa dan sedikit kelurahan, sedangkan Kota umumnya banyak kelurahan dan sedikit desa.

Cell 7: Pisahkan x dan y

drop membuang kolom nama, provinsi, dan jenis, sisanya (kecamatan, kelurahan, desa) menjadi fitur x. Kolom jenis menjadi target y. Tidak ada output.

Cell 8: Ubah ke array angka

Model hanya menerima angka, jadi LabelEncoder mengubah target teks menjadi angka. Outputnya 0 = Kabupaten dan 1 = Kota. Setelah itu x dijadikan array float64 dan y array int64.



Cell 9: Bagi data latih dan uji

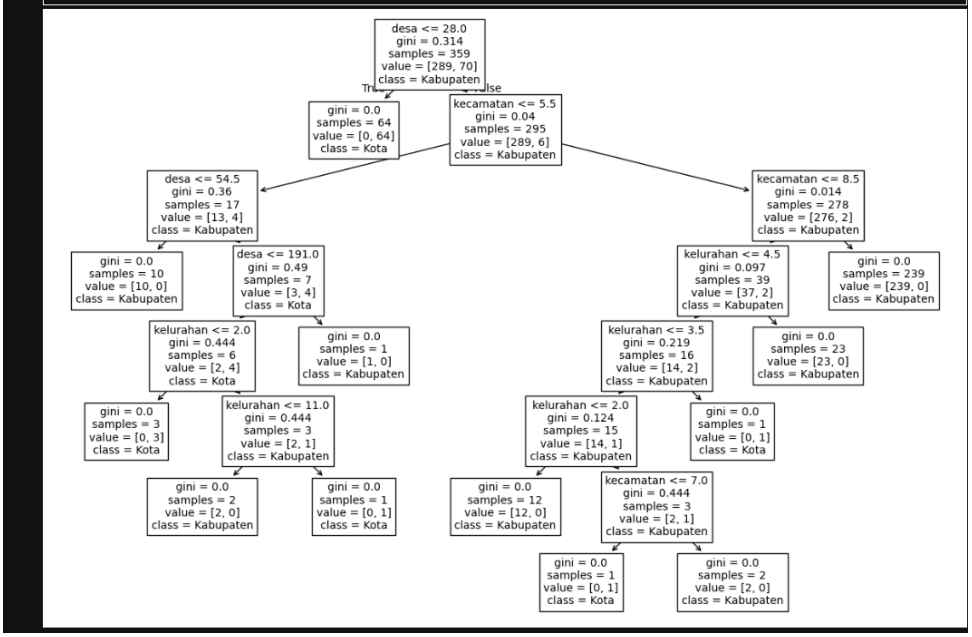
Data dibagi 70% untuk melatih dan 30% untuk menguji (test_size=0.3). random_state=0 membuat pembagiannya selalu sama. Data uji berisi 155 data (terlihat dari support pada report nanti). Tidak ada output.

Cell 10: Buat model dan latih

DecisionTreeClassifier() membuat model pohon keputusan, dan fit melatihnya dengan data latih untuk mencari aturan "jika-maka" yang membedakan kabupaten dan kota. Outputnya DecisionTreeClassifier(), tanda model sudah dilatih.

```
[11]: #Purnomo Jeremy Bambang_202431026
from sklearn import tree

plt.subplots(figsize = (15,10))
tree.plot_tree(classifier, feature_names=['kecamatan', 'kelurahan', 'desa'], class_names=le.classes_, fontsize = 10)
plt.show()
```



Cell 11: Visualisasi pohon

tree.plot_tree menggambar pohon aturan yang dipelajari model. Tiap kotak berisi syarat pemisah, gini (ketidakmurnian, 0 berarti satu kelas saja), samples (jumlah data), value (jumlah tiap kelas), dan class (kelas dominan). Dari gambar:

- Kotak teratas memakai syarat desa <= 28. Daerah dengan desa 28 atau kurang (64 data) semuanya Kota (gini = 0).
- Sisanya (295 data) hampir semuanya Kabupaten (289 Kabupaten, 6 Kota). Pohon lalu memakai kecamatan dan kelurahan untuk memisahkan sisa yang mirip.

Artinya jumlah desa adalah pembeda paling kuat antara kabupaten dan kota.

```
[12]: #Purnomo Jeremy Bambang_202431026
#data baru: [kecamatan, kelurahan, desa]
y_baru = classifier.predict([[6, 30, 0]])
print(y_baru)
print("Hasil prediksi:", le.inverse_transform(y_baru))
```

```
[1]
Hasil prediksi: ['Kota']
```

```
[13]: #Purnomo Jeremy Bambang_202431026
from sklearn.metrics import accuracy_score

y_pred = classifier.predict(x_test)
akurasi = accuracy_score(y_test, y_pred)
print(akurasi)
```

```
0.9419354838709677
```

```
[14]: #Purnomo Jeremy Bambang_202431026
from sklearn.metrics import classification_report

print(classification_report(y_test, y_pred, target_names=le.classes_))
```

```
              precision    recall  f1-score   support

   Kabupaten       0.97      0.96      0.96       127
        Kota       0.83      0.86      0.84        28

    accuracy                           0.94       155
   macro avg       0.90      0.91      0.90       155
weighted avg       0.94      0.94      0.94       155
```

Cell 12: Uji dengan 1 data baru

Data rekaan dengan 6 kecamatan, 30 kelurahan, dan 0 desa diprediksi memakai predict. Outputnya [1] dan Hasil prediksi: ['Kota']. Hasilnya masuk akal: daerah tanpa desa dan banyak kelurahan sesuai ciri kota.

Cell 13: Hitung akurasi

accuracy_score membandingkan prediksi dengan label asli pada data uji. Outputnya 0,9419, artinya sekitar 94,2% prediksi benar.

Cell 14: classification_report

Menampilkan precision, recall, dan f1-score tiap kelas.

- Kabupaten: precision 0,97, recall 0,96, f1 0,96 dari 127 data. Hampir semua kabupaten ditebak dengan benar.
- Kota: precision 0,83, recall 0,86, f1 0,84 dari 28 data. Kota ditebak sedikit kurang akurat, kemungkinan karena jumlah datanya jauh lebih sedikit.
- Akurasi keseluruhan 0,94 dari 155 data uji.

Secara umum model sudah baik membedakan kabupaten dan kota dari jumlah kecamatan, kelurahan, dan desa.

5. Naive Bayes

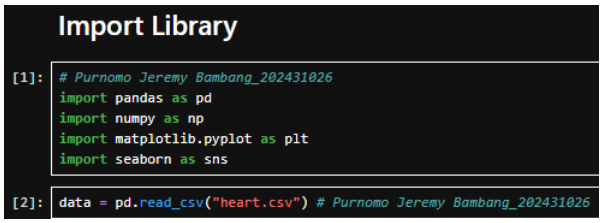
Judul dataset: Heart Failure Prediction Dataset

Link Kaggle: https://www.kaggle.com/datasets/fedesoriano/heart-failure-prediction?select=heart.csv

Jumlah baris × kolom: 918 baris × 12 kolom

Penjelasan data: Dataset ini berisi data kesehatan pasien, seperti umur, jenis kelamin, jenis nyeri dada, tekanan darah, kadar kolesterol, dan detak jantung maksimum. Targetnya adalah kolom HeartDisease (1 = menderita penyakit jantung, 0 = normal). Data ini dipakai untuk memprediksi apakah seorang pasien berisiko menderita penyakit jantung berdasarkan hasil pemeriksaannya.

Dataset ini berbeda dari dataset EDA (Titanic), jumlah barisnya melebihi 500, dan target berupa kategori (HeartDisease), fitur utamanya angka, dan ada beberapa fitur kategori (misalnya Sex dan ChestPainType).



Cell 1: Import library

Memanggil pandas dan numpy untuk mengolah data, matplotlib dan seaborn untuk grafik. Tidak ada output.

Cell 2: Baca dataset

pd.read_csv("heart.csv") membaca file CSV ke variabel data. Tidak ada output.

```
[3]: data.head() # Purnomo Jeremy Bambang_202431026
```

[3]:

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	40	M	ATA	140	289	0	Normal	172	N	0.0	Up	0
1	49	F	NAP	160	180	0	Normal	156	N	1.0	Flat	1
2	37	M	ATA	130	283	0	ST	98	N	0.0	Up	0
3	48	F	ASY	138	214	0	Normal	108	Y	1.5	Flat	1
4	54	M	NAP	150	195	0	Normal	122	N	0.0	Up	0

Cell 3: data.head()

Menampilkan 5 baris pertama. Setiap baris adalah satu pasien dengan data seperti umur, jenis kelamin, tekanan darah, kolesterol, dan HeartDisease sebagai target. Misalnya pasien pertama pria 40 tahun dengan HeartDisease = 0 (normal).

```
[4]: data.info() # Purnomo Jeremy Bambang_202431026
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 918 entries, 0 to 917
Data columns (total 12 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Age             918 non-null    int64
 1   Sex             918 non-null    object
 2   ChestPainType   918 non-null    object
 3   RestingBP       918 non-null    int64
 4   Cholesterol     918 non-null    int64
 5   FastingBS       918 non-null    int64
 6   RestingECG      918 non-null    object
 7   MaxHR           918 non-null    int64
 8   ExerciseAngina  918 non-null    object
 9   Oldpeak         918 non-null    float64
 10  ST_Slope        918 non-null    object
 11  HeartDisease    918 non-null    int64
dtypes: float64(1), int64(6), object(5)
memory usage: 86.2+ KB
```

```
[5]: data.describe() # Purnomo Jeremy Bambang_202431026
```

[5]:

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak	HeartDisease
count	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000	918.000000
mean	53.510893	132.396514	198.799564	0.233115	136.809368	0.887364	0.553377
std	9.432617	18.514154	109.384145	0.423046	25.460334	1.066570	0.497414
min	28.000000	0.000000	0.000000	0.000000	60.000000	-2.600000	0.000000
25%	47.000000	120.000000	173.250000	0.000000	120.000000	0.000000	0.000000
50%	54.000000	130.000000	223.000000	0.000000	138.000000	0.600000	1.000000
75%	60.000000	140.000000	267.000000	0.000000	156.000000	1.500000	1.000000
max	77.000000	200.000000	603.000000	1.000000	202.000000	6.200000	1.000000

Cell 4: data.info()

Menampilkan struktur data. Ada 918 baris dan 12 kolom, semuanya terisi penuh (tidak ada data kosong). Ada 6 kolom angka (int64 dan float64) dan 5 kolom teks (object).

Cell 5: data.describe()

Menampilkan statistik kolom angka. Rata-rata umur 53,5 tahun, rata-rata tekanan darah 132,4, dan rata-rata kolesterol 198,8. Rata-rata HeartDisease 0,55 berarti sekitar 55% pasien menderita penyakit jantung. Cholesterol dan RestingBP punya nilai minimum 0, yang tidak wajar dan kemungkinan tanda data tidak tercatat.

```
[6]: # Purnomo Jeremy Bambang_202431026
     # Mengambil kolom numerikal (target HeartDisease tidak dihitung sebagai fitur)
     numerical_features = [feature for feature in data.columns if pd.api.types.is_numeric_dtype(data[feature]) and feature != 'HeartDisease']
     print("Jumlah fitur numerik = ",
           len(numerical_features))
     print("Daftar fitur numerik = ",
           numerical_features)
     display(data[numerical_features].head())
Jumlah fitur numerik =  6
Daftar fitur numerik =  ['Age', 'RestingBP', 'Cholesterol', 'FastingBS', 'MaxHR', 'Oldpeak']
```

	Age	RestingBP	Cholesterol	FastingBS	MaxHR	Oldpeak
0	40	140	289	0	172	0.0
1	49	160	180	0	156	1.0
2	37	130	283	0	98	0.0
3	48	138	214	0	108	1.5
4	54	150	195	0	122	0.0

Cell 6: Fitur numerik

List comprehension memilih kolom bertipe angka, kecuali target HeartDisease. Outputnya 6 fitur numerik: Age, RestingBP, Cholesterol, FastingBS, MaxHR, dan Oldpeak, ditambah 5 baris contohnya.

```
[7]: # Purnomo Jeremy Bambang_202431026
     #Mengambil kolom kategorikal
     categorical_features = [feature for feature in data.columns if not pd.api.types.is_numeric_dtype(data[feature])]
     print("Jumlah fitur kategorikal = ",
           len(categorical_features))
     print("Daftar fitur kategorikal = ",
           categorical_features)
     display(data[categorical_features].head())

Jumlah fitur kategorikal =  5
Daftar fitur kategorikal =  ['Sex', 'ChestPainType', 'RestingECG', 'ExerciseAngina', 'ST_Slope']
```

	Sex	ChestPainType	RestingECG	ExerciseAngina	ST_Slope
0	M	ATA	Normal	N	Up
1	F	NAP	Normal	N	Flat
2	M	ATA	ST	N	Up
3	F	ASY	Normal	Y	Flat
4	M	NAP	Normal	N	Up

Cell 7: Fitur kategorikal

Cara yang sama, tapi memilih kolom bertipe teks. Outputnya 5 fitur kategorikal: Sex, ChestPainType, RestingECG, ExerciseAngina, dan ST_Slope, ditambah 5 baris contohnya.

```
[8]: # Purnomo Jeremy Bambang_202431026
     # Menampilkan data unik pada fitur kategorikal
     for i in categorical_features:
         print(i, ":", data[i].unique())

Sex : ['M' 'F']
ChestPainType : ['ATA' 'NAP' 'ASY' 'TA']
RestingECG : ['Normal' 'ST' 'LVH']
ExerciseAngina : ['N' 'Y']
ST_Slope : ['Up' 'Flat' 'Down']
```

```
[9]: # Purnomo Jeremy Bambang_202431026
     # Mengubah kolom kategorikal menjadi angka untuk klasifikasi
     from sklearn.preprocessing import LabelEncoder
     le = LabelEncoder()
     for feature in categorical_features:
         print(data[feature].unique())
         data[feature] = le.fit_transform(data[feature])
         print(data[feature].unique())

['M' 'F']
[1 0]
['ATA' 'NAP' 'ASY' 'TA']
[1 2 0 3]
['Normal' 'ST' 'LVH']
[1 2 0]
['N' 'Y']
[0 1]
['Up' 'Flat' 'Down']
[2 1 0]
```

Cell 8: Isi unik fitur kategorikal

Mencetak nilai unik tiap fitur kategorikal, yaitu Sex (M, F), ChestPainType (ATA, NAP, ASY, TA), RestingECG (Normal, ST, LVH), ExerciseAngina (N, Y), dan ST_Slope (Up, Flat, Down). Dari sini terlihat apa saja pilihan di tiap kolom sebelum diubah ke angka.

Cell 9: LabelEncoder

Mengubah tiap kolom teks menjadi angka, karena model hanya menerima angka. Outputnya menunjukkan nilai sebelum dan sesudah, misalnya Sex: M, F menjadi 1, 0, dan ST_Slope: Up, Flat, Down menjadi 2, 1, 0. Pengurutannya mengikuti abjad.

```
[10]: data.head() # Purnomo Jeremy Bambang_202431026
```

[10]:

	Age	Sex	ChestPainType	RestingBP	Cholesterol	FastingBS	RestingECG	MaxHR	ExerciseAngina	Oldpeak	ST_Slope	HeartDisease
0	40	1	1	140	289	0	1	172	0	0.0	2	0
1	49	0	2	160	180	0	1	156	0	1.0	1	1
2	37	1	1	130	283	0	2	98	0	0.0	2	0
3	48	0	0	138	214	0	1	108	1	1.5	1	1
4	54	1	2	150	195	0	1	122	0	0.0	2	0

Cell 10: data.head() setelah encoding

Menampilkan data setelah diubah. Semua kolom kini berupa angka, misalnya Sex M menjadi 1 dan ST_Slope Up menjadi 2.

```
[11]: # Purnomo Jeremy Bambang_202431026
      # Mengambil kolom untuk fitur dan label
      x = data.drop(['HeartDisease'], axis=1)
      y = data['HeartDisease']
      print(x)
      print(y)
```

```
      Age  Sex  ChestPainType  RestingBP  Cholesterol  FastingBS  RestingECG  \
0      40    1              1        140          289          0           1
1      49    0              2        160          180          0           1
2      37    1              1        130          283          0           2
3      48    0              0        138          214          0           1
4      54    1              2        150          195          0           1
..    ...  ...            ...        ...          ...        ...         ...
913    45    1              3        110          264          0           1
914    68    1              0        144          193          1           1
915    57    1              0        130          131          0           1
916    57    0              1        130          236          0           0
917    38    1              2        138          175          0           1

     MaxHR  ExerciseAngina  Oldpeak  ST_Slope
0      172               0      0.0         2
1      156               0      1.0         1
2       98               0      0.0         2
3      108               1      1.5         1
4      122               0      0.0         2
..     ...             ...      ...       ...
913    132               0      1.2         1
914    141               0      3.4         1
915    115               1      1.2         1
916    174               0      0.0         1
917    173               0      0.0         2

[918 rows x 11 columns]
0      0
1      1
2      0
3      1
4      0
      ..
913    1
914    1
915    1
916    1
917    0
Name: HeartDisease, Length: 918, dtype: int64
```

Cell 11: Pisahkan x dan y

x berisi fitur, yaitu semua kolom kecuali HeartDisease, sedangkan y berisi target HeartDisease yang ingin diprediksi. Keduanya dipisah karena model belajar dari x untuk menebak y. Outputnya x berupa tabel 918 baris × 11 kolom yang semuanya sudah angka, dan y berupa 918 nilai 0 (normal) atau 1 (penyakit jantung), satu untuk tiap pasien.

```
[12]: # Purnomo Jeremy Bambang_202431026
      # Membagi data menjadi data test dan data train
      from sklearn.model_selection import train_test_split as tts

      x_train, x_test, y_train, y_test = tts(x, y, test_size = 0.25, random_state = 0)
```

Cell 12: Bagi data latih dan uji

Data dibagi 75% untuk melatih dan 25% untuk menguji. random_state=0 membuat pembagiannya selalu sama. Data latih berjumlah 688 dan data uji 230. Tidak ada output.

```
[13]: # Purnomo Jeremy Bambang_202431026
      #melatih model Gaussian Naive Bayes
      from sklearn.naive_bayes import GaussianNB

      model = GaussianNB()
      model.fit(x_train, y_train)
```

```
[13]: GaussianNB
      ▶ Parameters
```

```
[14]: # Purnomo Jeremy Bambang_202431026
      y_pred_train = model.predict(x_train)
      y_pred_test = model.predict(x_test)
```

Cell 13: Buat model dan latih

GaussianNB() membuat model Naive Bayes dan fit melatihnya dengan data latih. Model mempelajari pola nilai fitur untuk tiap kelas. Outputnya GaussianNB(), tanda model sudah dilatih.

Cell 14: Prediksi data latih dan uji

predict menebak kelas untuk data latih dan data uji, hasilnya disimpan di y_pred_train dan y_pred_test. Tidak ada output.

```
[15]: # Purnomo Jeremy Bambang_202431026
      from sklearn.metrics import confusion_matrix, classification_report

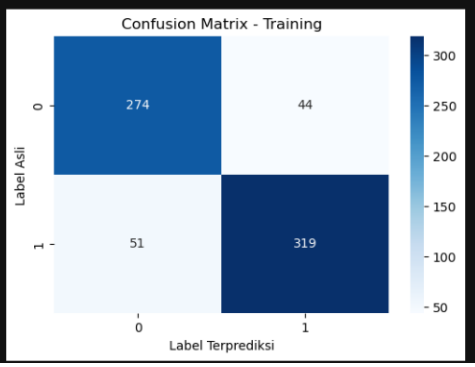
      print("Evaluasi Training")
      cm_train = confusion_matrix(y_train, y_pred_train)
      print(cm_train)
      print(classification_report(y_train, y_pred_train))

      plt.figure(figsize = (6, 4))
      sns.heatmap(cm_train, annot = True, fmt='d', cmap = 'Blues', xticklabels = np.unique(y), yticklabels = np.unique(y))
      plt.title("Confusion Matrix - Training")
      plt.xlabel("Label Terprediksi")
      plt.ylabel("Label Asli")
      plt.show()
```

```
Evaluasi Training
[[274  44]
 [ 51 319]]
              precision    recall  f1-score   support

           0       0.84      0.86      0.85       318
           1       0.88      0.86      0.87       370

    accuracy                           0.86       688
   macro avg       0.86      0.86      0.86       688
weighted avg       0.86      0.86      0.86       688
```



Cell 15: Evaluasi training

Outputnya confusion matrix dan report, serta heatmap berjudul “Confusion Matrix - Training”. Pada confusion matrix, diagonal adalah tebakan benar. Dari 688 data latih: 274 normal ditebak benar, 319 sakit ditebak benar, 44 normal salah ditebak sakit, dan 51 sakit salah ditebak normal. Akurasi training 0,86 (86%).

```
[16]: # Purnomo Jeremy Bambang_202431026
print("Evaluasi Testing")
cm_test = confusion_matrix(y_test, y_pred_test)
print(cm_test)
print(classification_report(y_test, y_pred_test))

plt.figure(figsize = (6, 4))
sns.heatmap(cm_test, annot = True, fmt='d', cmap = 'Greens', xticklabels = np.unique(y), yticklabels = np.unique(y))
plt.title("Confusion Matrix - Testing")
plt.xlabel("Label Terprediksi")
plt.ylabel("Label Asli")
plt.show()
```

```
Evaluasi Testing
[[ 73  19]
 [ 16 122]]
              precision    recall  f1-score   support

           0       0.82      0.79      0.81        92
           1       0.87      0.88      0.87       138

    accuracy                           0.85       230
   macro avg       0.84      0.84      0.84       230
weighted avg       0.85      0.85      0.85       230
```



Cell 16: Evaluasi testing

Langkah yang sama untuk data uji, dengan heatmap berjudul “Confusion Matrix - Testing”. Dari 230 data uji: 73 normal ditebak benar, 122 sakit ditebak benar, 19 normal salah ditebak sakit, dan 16 sakit salah ditebak normal. Akurasi testing 0,85 (85%). Selisihnya dengan akurasi training sangat kecil (86% vs 85%), jadi model tidak overfitting dan cukup baik pada data baru. Kelas sakit (1) sedikit lebih baik ditebak (f1 0,87) dibanding kelas normal (0).

```
[17]: # Purnomo Jeremy Bambang_202431026
# Uji dengan 1 data baru
data_baru = pd.DataFrame([[55, 1, 0, 140, 230, 0, 1, 120, 1, 1.5, 1]], columns = x.columns)
hasil = model.predict(data_baru)
print("Hasil prediksi:", hasil[0], "(1 = sakit jantung, 0 = normal)")
```

```
Hasil prediksi: 1 (1 = sakit jantung, 0 = normal)
```

Cell 17: Uji dengan 1 data baru

Satu data pasien rekaan dibuat dengan urutan kolom sama seperti x (nilainya sudah dalam bentuk angka hasil encoding), lalu diprediksi. Outputnya Hasil prediksi: 1, artinya model menebak pasien tersebut menderita penyakit jantung. Hasilnya masuk akal karena pasien rekaan ini punya angina saat olahraga dan ST slope Flat, dua ciri yang umum pada pasien penyakit jantung.